*.dtl

Purpose

A decision-table file holds one or more tables. Each table evaluates a set of conditions every time step and fires the matching action when an alternative is satisfied. SWAT+ uses decision tables to drive land-use management (planting, harvest, fertiliser, irrigation), reservoir releases, scenario land-use changes, and flow control.

The four decision-table files declared in `type input_condition` are:

- `lum.dtl`: land-use management actions.
- `res_rel.dtl`: reservoir release rules.
- `scen_lu.dtl`: scenario-driven land-use changes.
- `flo_con.dtl`: flow control rules.

All four use the same file format. They differ only in which decision-table array they fill (`dtbl_lum`, `dtbl_res`, `dtbl_scen`, `dtbl_flo`).

Source

- Readers: `src/dtbl_lum_read.f90`, `src/dtbl_flocon_read.f90` (and matching readers for reservoir-release and scenario tables).
- Type definitions: `src/conditional_module.f90`. Each table is a `decision_table` instance; its `cond`, `alt`, and `act` arrays hold conditions, alternatives, and actions.

File format

A single file holds `mdtbl` tables.

- Line 1: title (skipped).
- Line 2: `mdtbl`, the number of decision tables in the file.
- Line 3: blank line (skipped).

For each of the `mdtbl` tables:

- One header line (skipped).
- One name line: `name`, `conds`, `alts`, `acts` (table name, number of conditions, number of alternatives, number of actions).
- One header line for the conditions block (skipped).
- `conds` lines, one per condition. Each line names a conditional variable plus `alts` alternative thresholds. See Condition row below.
- One header line for the actions block (skipped).
- `acts` lines, one per action. See Action row below.

Condition row

Each condition row is read into a `condition_var` record with the following fields, then `alts` alternative thresholds are read into the `alt(ic, 1..alts)` array.

Fields on a condition row:

#	Field	Type	Description
1	var	char	conditional variable name (for example `land_use`, `plant_gro`, `phu_plant`, `soil_water`, `prob_unif`)
2	obj	char	object the variable applies to
3	obj_num	int	object number
4	lim_var	char	limit-variable name
5	lim_op	char	comparison operator
6	lim_const	real	comparison constant
7+	alt(1..alts)	char	one alternative threshold per alternative column

When `var == "prob_unif"`, the reader rereads the row and treats the second token as the probability fraction `frac_app` rather than the standard condition layout.

Action row

#	Field	Type	Description
1	act_typ	char	action type (for example `plant`, `harvest`, `fert`, `irrigate`, `lu_change`)
2	obj	char	target object
3	obj_num	int	target object number
4	name	char	action name pointing into the matching ops or input database
5	option	char	option tag
6	const	real	numeric constant for the action
7	const2	real	second numeric constant
8	fp	char	flag or pointer
9+	act_hit(1..alts)	char	one outcome flag per alternative column

Notes

- The reader allocates `cond(conds)`, `alt(conds, alts)`, `act(acts)`, and `act_hit(alts)` based on the header values. A mismatch between declared counts and actual rows truncates or overruns these arrays.
- After all tables are read, `lum.dtl` does a second pass to count HRUs whose `land_use_mgt_c` matches any `land_use` condition. The counts feed probabilistic operations.

Example

Top of `refdata/Ames_sub1/lum.dtl`:

```
lum.dtl: written by SWAT+ editor for SWAT+ rev.60.5.7
8

corn_grain
name                 conds   alts   acts
pl_hv_corn               2      2      2
var                   obj   obj_num   lim_var   lim_op   lim_const   alt1   alt2
plant_gro            plant         1   tree     >        0           y      n
phu_plant            plant         1                              >  1.0    n      y
act_typ              obj   obj_num   name                option   const   const2   fp   outcome1   outcom
plant                hru   0         corn                below    0       0        y    y          n
harvest_kill         hru   0         harv_grain_corn     now      0       0        y    n          y
```

Reading this:

- File holds 8 tables. The first is named `pl_hv_corn` with 2 conditions, 2 alternatives, 2 actions.
- Condition 1 checks `plant_gro` on plant object 1 against `tree > 0`. Alt 1 fires when `y`, alt 2 fires when `n`.
- Condition 2 checks `phu_plant > 1.0`. Alt 1 fires when `n`, alt 2 fires when `y`.
- Action 1: `plant` HRU operation named `corn` with option `below`. Fires under alt 1, not alt 2.

- Action 2: `harvest_kill` HRU operation named `harv_grain_corn` with option `now`. Fires under alt 2, not alt 1.

The exact column layout varies a little between datasets because the reader is fully list-directed.

Related files

- `landuse.lum`. Plant and management names referenced by actions.
- `*.ops`. Operation parameter databases that the actions resolve `name` against.
- `management.sch`. Fixed schedules used alongside decision tables.